This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2025.3565675

Multi-objective Competitive Co-evolutionary Optimization and Regularity-based Decision-making for Two-agent Wargame Strategy Optimization

Ritam Guha*, Ryan Mckendrick†, Bradley Feest†, and Kalyanmoy Deb*
*Michigan State University, East Lansing, MI 48824, U.S.A. {guharita, kdeb}@msu.com
†Northrop Grumman, Falls Church, VA 22042, U.S.A. {ryan.mckendrick, bradley.feest}@ngc.com

***Abstract*— Many practical problems involve multiple interdependent agents, each aiming to optimize its own objectives. Wargame strategy optimization, which requires optimizing strategies for at least two agents—attackers and defenders—presents unique challenges due to the interdependence of the agents' strategies. This characteristic necessitates a co-evolutionary approach, where each agent's strategy is continually adjusted in response to the other's. The complexity increases when each agent pursues multiple conflicting objectives, resulting in Pareto-optimal strategy sets that require sequential decision-making (DM). To address these challenges, we introduce a novel multi-objective competitive co-evolutionary optimization (MoC-CoEv) framework, specifically tailored for wargame strategy optimization. This framework integrates regularity-based search with an iterative and interactive DM approach, fostering a continuous interplay between co-evolving agents. Additionally, we introduce the concept of progressive shrinking, which interactively reduces the dimensions of the agents' strategy parameters to mirror real-world decision-making by enforcing commitment to earlier moves and facilitating effective strategic choices. Our flexible and adaptable framework also supports alternative strategies, such as deception, and can be applied to other multi-agent optimization problems.**

***Index Terms*—Attacker-defender systems Co-evolution Multi-objective games Multi-agent systems Decision-making**

I. Introduction

Many real-world problems involve multiple agents, each aiming to maximize its own goals while simultaneously minimizing the opponent's goals, particularly in scenarios involving defense and attack. In these problems, the strategy of one agent must be combined with the strategy of the opponent to evaluate its own goals. This interaction between the agents' strategies gives rise to competitive co-evolutionary (C-CoEv) optimization problems. One such example is the wargame strategy optimization problem.

Wargame strategy optimization is a critical component of military strategy and preparation. History has shown that superior firepower alone does not guarantee victory. Much of the outcome depends on how each side utilizes its resources and develops effective strategies to counter the opponent's movements. It is important to note that this paper does not support or glorify war; rather, it presents wargame strategy optimization as a challenging application scenario, resulting in a multi-agent C-CoEv optimization problem. Additionally, wargame simulations [1]–[3] have gained substantial traction in the gaming community. In essence, wargame strategy optimization seeks to identify one or more strategies that optimize certain goals or objectives defined by the participating agents. In the simplest wargame setting, two agents are involved: the offense and the defense. The offense agent attacks the defense base using its military resources, while the defense agent attempts to protect itself from the offense's attacks.

Each agent has its own set of parameters relating to various resources, such as logistical data, asset numbers, and more, which can be adjusted to formulate a response strategy against the opponent. As noted, the outcome of any wargame simulation depends on the strategies adopted by both agents. The effectiveness of one agent's strategy is significantly influenced by the strategy chosen by the other agent. Thus, optimizing one agent's strategy in isolation is not feasible, as is often done in standard optimization problems. Since both agents aim to optimize their strategies while responding to one another, there is a need for both agents to co-evolve. As one agent's strategy improves through optimization iterations, the other agent simultaneously learns to enhance its own strategy. This intertwined optimization process results in a co-evolutionary optimization problem. When both agents have conflicting objectives, the problem becomes more complex and is referred to as a multi-objective competitive co-evolutionary (MoCCoEv) problem.

In recent years, co-evolutionary algorithms have attracted significant attention in multi-agent systems [4]–[6]. The concept of co-evolution is inspired by nature, where different species evolve through interactions, with survival and success driven by co-evolution [7]. Co-evolutionary processes can be broadly classified into three categories: cooperative, competitive and *coopetitive*. Cooperative co-evolution [8], [9] occurs when multiple species evolve by cooperating to achieve their respective goals. In contrast, competitive co-evolution [10], [11] arises when species evolve by competing to achieve their individual goals. A detailed survey of these co-evolutionary studies can be found in [12]. Another paradigm, known as coopetition, involves species collaborating in some areas while competing in others [13]. In wargame strategy optimization, the goals of the two agents are generally in conflict, making the task a competitive co-evolutionary problem.

Each agent in a co-evolutionary setup has decision variables that it can control for its actions. However, the performance (or fitness) of an agent's chosen variable setup depends on the

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2025.3565675

strategy of the other agent. This distinguishes co-evolutionary optimization algorithms from traditional single-population evolutionary algorithms [14]. While co-evolutionary algorithms are often proposed for single-objective optimization [15], [16], in practical multi-agent systems, such as wargame strategy optimization, each agent may have multiple conflicting objectives. The objectives of one agent can differ entirely from those of the other agent, and even when objectives are similar, they may be opposing. For instance, the defense agent may seek strategies that minimize the loss of its assets, while the offense would likely aim to maximize the damage to the defense's assets.

There is a growing body of literature on multi-objective co-evolution. For instance, in [17], the authors introduced co-evolution in multi-objective optimization, where solutions act as learners and objectives serve as teachers, providing gradients or challenges to distinguish learners based on their performance. This leads to co-evolution between populations of teachers and learners. However, there is no interactive decision-making (DM) involved in the process. The paper does not discuss how a learner will be selected at the end. A similar approach is explored in [18], where the goal is to find an ideal set of evaluators that can improve the distinction among learners. Not only does the co-evolution works on a cooperative environment, it also does not involve any DM task. In [11], Meneghini et al. introduced competitive co-evolution as a method to enhance robustness in multi-objective optimization. This approach utilizes two populations: one for solutions and the other for perturbations to the solution set. For each solution, multiple perturbations are considered, and a scalarized function is employed to identify the worst-case objectives of the solution. Finding a Pareto front (PF) of perturbations or any decision-making (DM) activities following the identification of the robust front were not the goal of the study, which contrasts with the focus of the current paper. Recently, Benford et al. [19] demonstrated the use of co-evolutionary algorithms to balance worst-case and average-case performances, thereby enhancing solution robustness. In their study, the authors address a single multi-objective optimization problem using a co-evolutionary framework, where the two objectives are average-case and worst-case performance, making the approach difficult to generalize. Furthermore, their paper does not discuss any subsequent decision-making (DM) procedures. From a theoretical perspective, studies like [20] by Lehre et al. on the runtime analysis of co-evolutionary algorithms, or [21] by Sim et al. on treating co-evolution from an evolutionary game theory perspective, have provided evidence for the benefits of using co-evolutionary approaches. Despite these advances, no study had considered a comprehensive integration of the following essential tasks needed to solve a true multi-objective competitive co-evolutionary optimization and decision-making problem from practice: (i) finding multiple agent-wise Pareto-optimal fronts, (ii) an iterative and interactive decision-making task in tandem involving both agents' variables at the same level, (iii) demonstrating clearly the trade-offs among individual agent's objectives in its own PFs and demonstrating clearly the trade-offs among both agents' PFs, (iv) incorporating restrictions in decision-making to make the DM approach practical, (v) clearly demonstrating how the co-evolutionary process ends up with an equilibrium bi-agent strategy pair or an equilibrium bi-agent cycle of strategies, and (vi) clearly illustrating all the above on a real-world problem.

In this paper, we propose an integrated framework that combines various practical aspects of co-evolutionary optimization and decision-making, building on recent studies in MoCCoEv [22], [23]. Subsequently, in [24], a MoCCoEv framework along with a preliminary decision-making (DM) approach was proposed for the first time to address wargame strategy optimization. This method had limitations, such as selecting strategies without considering the commitment to earlier moves, which reduced its practicality. Moreover, the paper performed a post-optimality analysis of the final population of strategies, which indicated the convergence of certain wargame variables. However, this concept was never utilized to effectively identify better solutions for the problem. To improve practicality and robustness of the previous procedures, here, we introduce a gradual reduction in problem size coupled with a re-optimization process, which we refer to as *progressive shrinking*. This approach ensures that agents maintain continuity with their consecutive committed moves, thereby enhancing the overall practicality of the framework. We highlight below the major contributions of this study compared to the previous studies:

- **Incorporation of iterative and interactive decision-making approach:** Earlier MoCCoEv studies considered a naive DM strategy, whereas, the wargame strategy optimization problem demands a bi-agent interactive DM approach applied sequentially in an iterative manner. We clearly demonstrate that such an iterative and interactive DM approach can lead to a systematic equilibrium point or a cycle of two-agent strategies.
- **Detailed analysis of computational models:** This paper offers a comprehensive analysis of three computational models for solving the wargame strategy optimization problem, each progressively being more practical. Two of these models are implemented in this study, while the third one is planned for future research.
- **Introduction of progressive shrinking to MoCCoEv for pragmatic decision-making task:** We introduce a novel progressive shrinking technique within the MoCCoEv framework. This technique enables more informed decision-making by progressively reducing the problem's dimensions, allowing agents to commit to their moves while staying adaptable to new strategies.
- **In-depth discussion of selected strategies:** The proposed framework is applied to a real-world wargame simulation, demonstrating its effectiveness in generating robust and intelligent strategies, which are further validated through expert analysis.

The rest of the paper is organized as follows: Section II describes the wargame strategy optimization problem, detailing the structure, objectives, and the complexity of the MoCoEv optimization task. This section also outlines potential computational models for solving the problem. Background on MoCCoEv and the proposed framework, including updates

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2025.3565675

to the optimization algorithm and customized post-optimal DM approaches, is provided in Section III. The results obtained from applying the proposed framework to the wargame strategy optimization problem are discussed in Section V, where the resulting solution is found to offer meaningful and intelligent wargame strategies supported by human experts. Finally, the paper concludes in Section VI. From here onward, we use the terms 'attacker' for offense and 'defender' for defense interchangeably.

II. Wargame Strategy Optimization Problem (WSOP)

Wargame strategy optimization problems (WSOPs) are highly idiosyncratic, and no single computational model can fully represent all types of wargames. In this paper, we focus on a wargame involving an attacker and a defender. The attacker aims to inflict as much damage as possible on an airbase (air-to-ground missile hits), while incurring minimal cost (monetary expenses and loss of attacker lives), all the while imposing the greatest cost on the defender (monetary expenses and loss of defender lives). The defender's goals are directly opposed to those of the attacker. Both agents' strategies can be divided into three subsets: research and engineering (RE), force composition (FC), and mission plans (MP). RE and FC decisions incur upfront monetary costs and are considered fixed once the mission begins. RE decisions define the capabilities of assets (e.g., stealth, speed, sensor ranges, and payload capacities), while FC decisions specify the number of assets brought into the conflict across different roles (e.g., strike assets and electronic warfare assets). MP decisions govern how assets coordinate and behave during the conflict (e.g., flight formations, rules of engagement, and routes). In total, the attacker has 50 variables to manipulate, while the defender has 24 variables.

Mathematically, the overall problem can be defined as follows:

Offense Goals:

$$\begin{aligned} \text{Minimize}_{\boldsymbol{x}} \quad & (-\text{OH}(\boldsymbol{x},\boldsymbol{y}), (\text{OE}(\boldsymbol{x},\boldsymbol{y}) - \text{DE}(\boldsymbol{x},\boldsymbol{y}))), \\ \text{subject to} \quad & x_i \in \{x_i^{(1)}, x_i^{(2)}, \ldots, x_i^{(c_i)}\},\ i = 1, \ldots, n_1. \end{aligned} \tag{1}$$

Defense Goals:

$$\begin{aligned} \text{Minimize}_{\boldsymbol{y}} \quad & (\text{OH}(\boldsymbol{x},\boldsymbol{y}), -(\text{OE}(\boldsymbol{x},\boldsymbol{y}) - \text{DE}(\boldsymbol{x},\boldsymbol{y}))), \\ \text{subject to} \quad & y_j \in \{y_j^{(1)}, y_j^{(1)}, \ldots, y_j^{(c_j)}\},\ j = 1, \ldots, n_2. \end{aligned} \tag{2}$$

where OH, OE, and DE represent *OffenseHits*, *OffenseExpenditures*, and *DefenseExpenditures*, respectively; x_i and y_j denote the values of the ith offense and j^{th} defense variables, respectively; and c_i and c_j represent the number of possible categorical values for x_i and y_j, respectively. The values for hits and expenses are typically derived from a high-fidelity simulation of the wargame scenario. However, in this study, we use surrogate models to approximate these indicators during the optimization process. Details about the construction of the surrogate models from the training data collected from the simulator are provided in the supplementary material.

A detailed description of a generic wargaming process is provided by the Norwegian Defence Research Establishment [25]. Schwartz et al. offer an in-depth discussion of AI-enabled wargaming [26], where a genetic algorithm (GA) is employed to optimize the wargame's course of action (COA). While our approach follows a similar direction, it introduces key distinctions. Specifically, our method addresses multiple objectives within a co-evolutionary framework, whereas Schwartz et al.'s approach operates with a single objective in a non-co-evolutionary setting, assuming a static opponent strategy. In contrast, our method accounts for a dynamic opponent, incorporating adaptability for the opponent to concurrently enhance its strategies.

A. Possible Computational Models

We tackle the WSOP by employing multiple computational models to address its complexities. In this paper, we present results for two distinct approaches, while outlining a third approach for future exploration. We assume that both agents are aware of the potential set of strategies of their opponents. This can be viewed as one agent's best approximation of the other's strategies, which is essential for the analysis. During the DM step, both agents evaluate all possible moves of their own against all possible moves of the other agent to determine the most suitable strategy. Although this assumption does not fully align with real-world scenarios, it remains a challenging task to find optimal strategies within a multi-objective and competitive co-evolutionary setup and to select an appropriate strategy accordingly. Playing the game optimally, with each agent acting in its best interest, can provide valuable insights into the consequences of different actions and enhance understanding of the intricacies of the wargame. Two of the computational models that can be used to represent the WSOP are described below.

1. **Competitive co-evolutionary optimization and offline decision-making:** The first step in solving the WSOP is to apply an optimization algorithm to identify multiple alternative Pareto strategies that are suitable for the problem at hand. Since the WSOP involves two agents with conflicting objectives, a standard multi-objective optimization algorithm cannot be used. Therefore, a competitive co-evolutionary framework (MoCCoEv) is adapted to find optimal strategies for both interconnected agents offline. In this process, we maintain two populations—one for each agent. All strategies in one population are combined with all strategies in the other population, as illustrated in Figure 1. An aggregation function is then applied to combine the objective values for each strategy. Based on these aggregated values, non-dominated sorting is performed to identify the Pareto strategies for each agent. The MoCCoEv procedure generates these Pareto strategies through combined and comprehensive evaluations. The goal of the MoCCoEv process is to progressively push these Pareto fronts closer to their individual objectives over successive generations.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2025.3565675

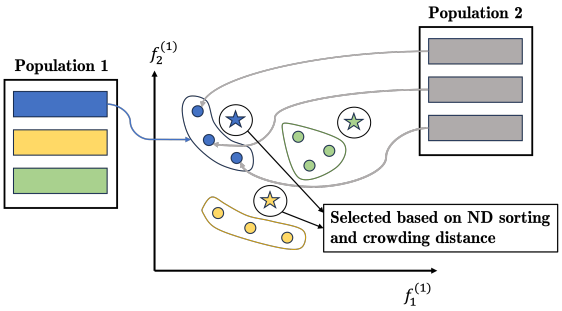


Fig. 1: Aggregated objective values for strategies of two populations

After the Pareto strategies are identified offline, the next step is to conduct a DM analysis for both agents, typically starting with one of the defense's Pareto strategies. This analysis proceeds by selecting the most appropriate responsive Pareto strategy at each iteration until the process converges to a final equilibrium strategy pair or a strategy cycle between the two agents. During the DM process, no new optimization runs are performed; both agents limit their selections to their respective Pareto strategies corresponding to the opponent's selected strategy from the initial step. Restricting an agent's decisions to the obtained Pareto sets ensures that each agent uses one of the best possible strategies identified in the combined search space.

Thus, in this computational model, a multi-objective competitive co-evolutionary optimization is first applied to find individual Pareto strategy sets for both agents offline. Subsequently, an offline simulation of repetitive multi-criterion decision-making (MCDM) moves is conducted to determine the final outcome. It is noteworthy that the tandem DM procedure (described in more detail in Section IV-A) can also be applied online in a computationally efficient manner, as the Pareto strategies for each agent were already determined offline—this will be discussed further in the future work, though not implemented for the results in this paper. The procedure of offline competitive co-evolution and offline DM is illustrated using a schematic diagram in the supplementary material.

2. **Progressive shrinking of *regular* MoCCoEv and iterative decision-making:** The procedure outlined above restricts each agent's choice to its offline-obtained Pareto set of strategies. As a result, at each DM step, any Pareto strategy can be selected, regardless of whether transitioning from the strategy chosen in the previous step is feasible. To make the DM process more practical, subsequent choices must account for decisions made in previous steps and be constrained to those Pareto strategies that allow for a smooth transition. For example, if a strategy is initiated with a type-1 fighter jet, it may not be feasible to switch to a strategy involving a type-2 fighter jet. The concept of progressive shrinking-based DM stems from this practical implementation scenario.

The progressive shrinking approach may present challenges with a finite-sized Pareto set of strategies. When a specific strategy is chosen by one of the agents (e.g., the offense), this may involve deploying certain types and numbers of aircraft with specific types of ammunition. In response to the defense's move, the offense cannot suddenly switch to a completely different strategy in the next step, such as following up with different types of ammunition that the aircraft may not be carrying as dictated by the previous move. Given the finite set of Pareto strategies identified during the optimization task, in the worst-case scenario, there may not be any Pareto strategy that maintains continuity with the previous decision. In such a situation, it may be necessary to fix certain problem variables for the offense to ensure continuity in the DM process and then reapply the MoCCoEv optimization on the reduced set of variables. Thus, progressive shrinking refers to a process of iterative optimization and shrinking of the variable space of the participating agents.

In the first progressive shrinking run, we fix the variables that are necessary to ensure continuity with previous moves. Subsequently, we continue to perform multiple progressive shrinking runs to reduce the dimensionality of the process, thereby better imitating realistic scenarios where the decisions are made in an iterative fashion. Various methods can be used to identify the fixed variables, and in this work, we propose a variable-wise statistical sensitivity approach to determine which variables should be fixed for the next MoCCoEv optimization run. After the initial defense-offense moves are made, we identify the variables for both agents that must be fixed to maintain continuity. Then, we apply MoCCoEv optimization with the reduced set of variables for both agents.

In later stages, the fixed variables are identified using coefficient of variation (CoV)-based statistical analysis. The interactive optimization and DM continue until a stopping criterion is met. More details about this iterative DM process are provided in Section IV-B. This process of fixing certain variables is analogous to finding a set of *regular* solutions, as proposed in recent studies [27]–[29]. Regular solutions exhibit common properties, reflecting regularity in the solutions that emerge as patterns within the obtained Pareto solutions, rather than being predefined. Shrinking the number of variables reduces the dimensionality of the search space and enables MoCCoEv to run faster than previous optimization runs. The progressive shrinking procedure is also illustrated using a schematic diagram in the supplementary material.

III. Multi-objective Competitive Co-evolutionary (MoCCoEv) Optimization

Multi-objective co-evolutionary optimization involves two or more agents, each of which has at least two conflicting objectives to optimize. Due to the conflicting nature of the objectives, instead of a single optimal solution, a set of PO solutions is obtained for each agent. In this section, we describe the WSOP at both the agent level as a single-agent multi-objective optimization task and at the wargame level as a multi-agent multi-objective optimization task.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2025.3565675

A. Single-agent Multi-objective Optimization

As highlighted earlier, WSOPs often have multiple conflicting objectives for each agent. Therefore, when optimized, each agent will have a set of trade-off PO strategies, from which a single preferred strategy must be selected through a subsequent DM task. It is important to note that a multi-objective optimization problem results in a PF, which consists of multiple PO solutions, each of which is equally optimal in the objective space. Thus, additional DM is required to choose the most suitable implementable strategy.

Multi-objective optimization problems can be solved using two broad procedures: (i) generational procedures, and (ii) simultaneous procedures. Generational procedures generate a single PO solution in one run by scalarizing multiple objectives into one [30], [31]. On the other hand, simultaneous procedures employ a population-based search to identify multiple PO solutions in a single run, thus reducing overall computational effort [32], [33]. For this reason, evolutionary multi-objective optimization (EMO) approaches are extensively used to handle real-world optimization problems [34], [35].

NSGA-II [36] is one of the most popular simultaneous multi-objective algorithms in the literature. In this study, we use a customized version of NSGA-II to handle the objectives of the WSOP under consideration.

B. Multi-agent Multi-objective Co-evolution

Without loss of generality, we consider two co-evolving agents (the attacker and the defender) in this study, although the approach can be extended to more than two agents. Problems P_1 (for the attacker) and P_2 (for the defender) involve variable vectors $\boldsymbol{x}$ and $\boldsymbol{y}$ of sizes N_1 and N_2, respectively. For the problem context, we refer to them as the specific strategies of P_1 and P_2, respectively. The objective and constraint functions for problem P_i are denoted as $\boldsymbol{f}^{(i)}(\boldsymbol{x},\boldsymbol{y})$ of size M_i and $\boldsymbol{g}^{(i)}(\boldsymbol{x},\boldsymbol{y}) \leq \boldsymbol{z}^{(i)}$ of size J_i. The optimization problem formulation is given below:

Problem P_1:

$$\begin{aligned} \text{Minimize}_{\boldsymbol{x}} \quad & \left(f_1^{(1)}(\boldsymbol{x},\boldsymbol{y}),\ldots,f_{M_1}^{(1)}(\boldsymbol{x},\boldsymbol{y})\right), \\ \text{subject to} \quad & g_j^{(1)}(\boldsymbol{x},\boldsymbol{y}) \leq 0,\ j=1,\ldots,J_1, \\ & x_i^{(L)} \leq x_i \leq x_i^{(U)},\ i=1,\ldots,n_1. \end{aligned} \tag{3}$$

Problem P_2:

$$\begin{aligned} \text{Minimize}_{\boldsymbol{y}} \quad & \left(f_1^{(2)}(\boldsymbol{x},\boldsymbol{y}),\ldots,f_{M_2}^{(2)}(\boldsymbol{x},\boldsymbol{y})\right), \\ \text{subject to} \quad & g_j^{(2)}(\boldsymbol{x},\boldsymbol{y}) \leq 0,\ j=1,\ldots,J_2, \\ & y_j^{(L)} \leq y_j \leq y_j^{(U)},\ j=1,\ldots,n_2. \end{aligned} \tag{4}$$

Note that P_1 and P_2 may have different numbers of objectives, i.e., $M_1 \neq M_2$. The key criterion for multi-objective competitive co-evolution is that both problems are multi-objective in nature, and the optimization objectives of P_1 should exhibit some direct or indirect conflict with those of P_2. In the context of multi-objective competitive co-evolution, a solution pair $(\boldsymbol{x}^*,\boldsymbol{y}^*)$ is said to be *Pareto optimal* if there does not exist another solution pair $(\boldsymbol{x}',\boldsymbol{y}')$ such that:

$$\begin{aligned} & f_i^{(1)}(\boldsymbol{x}',\boldsymbol{y}') \leq f_i^{(1)}(\boldsymbol{x}^*,\boldsymbol{y}^*) \quad \forall i \in \{1,2,\ldots,M_1\}, \\ & \quad \exists j \in \{1,2,\ldots,M_1\}, \quad f_j^{(1)}(\boldsymbol{x}',\boldsymbol{y}') < f_j^{(1)}(\boldsymbol{x}^*,\boldsymbol{y}^*), \\ & f_k^{(2)}(\boldsymbol{x}',\boldsymbol{y}') \leq f_k^{(2)}(\boldsymbol{x}^*,\boldsymbol{y}^*) \quad \forall k \in \{1,2,\ldots,M_2\}, \\ & \quad \exists l \in \{1,2,\ldots,M_2\}, \quad f_l^{(2)}(\boldsymbol{x}',\boldsymbol{y}') < f_l^{(2)}(\boldsymbol{x}^*,\boldsymbol{y}^*), \end{aligned} \tag{5}$$

where $\boldsymbol{x}^*,\boldsymbol{y}^*$ are the solution vectors for problems P_1 and P_2, respectively, and $f_i^{(1)}$ and $f_k^{(2)}$ represent the objective functions for each agent.

In a multi-objective scenario for each agent, it is also expected that the objectives of each agent are in conflict with each other, resulting in a PO solution set for each agent. However, since the evaluation of each objective function depends on a variable vector from both problems, optimizing each multi-objective optimization problem independently is neither fair nor practical, as it would imply that one agent can adjust the opponent's strategy while the other cannot, making the game unfair. If we optimize P_1 independently by including $\boldsymbol{y}$ in its variable set, the resulting PO front would likely be better than the PO front of the MoCCoEv problem P_1. Similarly, the same holds true for independently optimizing P_2. The inter-dependence of both agents requires a co-evolutionary search and optimization algorithm to simultaneously solve the problem and find a compromised PO front for each agent. We will discuss the MoCCoEv algorithm in the next subsection.

1) Multi-objective Competitive Co-evolutionary (MoCCoEv) Optimization Algorithm: In multi-objective co-evolutionary (MoCoEv) problems, Population 1 corresponds to a set of final solutions ($\boldsymbol{Z}^{(1)} = (\boldsymbol{z}^{(1),i},\ i=1,\ldots,N_1)$) and Population 2 corresponds to another set of final solutions ($\boldsymbol{Z}^{(2)} = (\boldsymbol{z}^{(2),i},\ i=1,\ldots,N_2)$). In a competitive MoCoEv problem, the solutions of the two problems are expected to be far from each other. To find and maintain these two sets of solutions, each population must not only focus on its best solutions but must also take into account the quality of solutions from the other population.

A pseudo-algorithm for the MoCCoEv procedure is provided in Algorithm 1. The algorithm begins with random or previously known solutions for both populations. Then, each member of one population is evaluated against all members of the other population, and an average fitness value is computed for each objective function. In this context, we assume that the opponent's population of strategies represents the range of possible responses to the current strategy. Although, in practice, the opponent's actual strategies may be unknown, this assumption is necessary to construct a viable computational model. Thus, the k-th population member of P_1, $\boldsymbol{x}^{(1),k}$, is paired with every member $\boldsymbol{y}^{(2),l}$ ($l=1,\ldots,N_2$) of Population P_2 one at a time, and N_2 objective vectors are evaluated. The mean fitness value of the i-th objective function of the k-th member of Population 1 is computed as follows:

$$F_i(\boldsymbol{x}^{(1),k}) = \frac{1}{N_2}\sum_{l=1}^{N_2} f_i(\boldsymbol{x}^{(1),k},\boldsymbol{y}^{(2),l}). \tag{6}$$

The same process is repeated for the second population

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2025.3565675

members.

Subsequently, at iteration t, evolutionary operations (selection, recombination, and mutation) are performed on $Pop_1^{(t-1)}$ using its fitness vectors and constraints, and a new population $Pop_1^{(t)}$ of size N_1 is created. These operations follow the NSGA-II algorithm. A total of τ_1 iterations are carried out for P_1 before the next population update for consecutive τ_2 iterations for P_2. The process continues until a termination condition is met.

Algorithm 1: A pseudo-code for the multi-objective co-evolutionary algorithm used in this study.

Input: Population sizes N_1 and N_2 for Pop_1 and Pop_2, number of generations T, number of consecutive generations (τ_1 and τ_2) assigned for each population update

Output: Final populations $Pop_1^{(T_1)}$ and $Pop_2^{(T_2)}$

```
Initialize populations Pop_1^(0) and Pop_2^(0)
gen ← 1
t_1 ← 0, t_2 ← 0
while gen ≤ T do
    for t = t_1 + 1 to t_1 + τ_1 do
        Generate offspring Q_1^(t) for Pop_1^(t−1) using selection and variation operators
        Merge Pop_1^(t) = Pop_1^(t−1) ∪ Q_1^(t)
        Evaluate Pop_1^(t) with all members of Pop_2^(t_2)
        Survive elite population of Pop_1^(t) of size N_1
    end
    t_1 ← t_1 + τ_1
    for t = t_2 + 1 to t_2 + τ_2 do
        Generate offsprings Q_2^(t) for Pop_2^(t−1) using selection and variation operators
        Merge Pop_2^(t) = Pop_2^(t−1) ∪ Q_2^(t)
        Evaluate Pop_2^(t) with all members of Pop_1^(t_1)
        Survive elite population of Pop_2^(t) of size N_2
    end
    t_2 ← t_2 + τ_2
    gen ← gen + τ_1 + τ_2
end
```

IV. Decision-making Procedures

Decision-making involving two interrelated PO sets controlled by independent agents is a challenging task and is often problem-dependent. First, an agent must select a strategy from its own PF, against which the opponent will choose one of many strategies from its own PF. This introduces uncertainty in the DM process for any multi-objective multi-agent system. In addition to the uncertainty arising from the other agent's choices at a specific iteration, the selection of a single preferred strategy from one's own PO set also depends on the strategy chosen by the opponent in the previous iteration. This makes the decisions of an agent heavily dependent on multiple iterations of the opponent's choices.

A. Decision-making using Offline Pareto Sets

The concept of MoCCoEv and the DM process proposed in this paper is as follows: The final Pareto strategies for both populations can be determined offline using available computational resources. Once these Pareto strategies are obtained, agents can iteratively select an appropriate preferred strategy from their respective Pareto sets. In the context of the attacker-defender WSOP, the defender may start by choosing a strategy from its Pareto strategy set based on initial preference information among its objectives. With the defender's strategy revealed, the attacker can then select the most preferred strategy from its own Pareto set to maximize the impact against the defender's chosen strategy. The defender then responds by selecting the next appropriate Pareto strategy from its set to counter the attacker's choice. This iterative process continues until an equilibrium state is reached—either a convergence to a specific attacker-defender strategy pair or a cycle of strategy sets.

One of the remaining tasks is to discuss the specific MCDM method that each agent can adopt to pick a single appropriate strategy from its own Pareto strategy set. Suppose there are N_1 attacking strategies and N_2 defending strategies after the MoCCoEv run is executed.

The first step of the DM process is to select an agent to start the game. Let us assume the defending agent is chosen first. The defense needs to select one strategy from the final N_2 strategies. Each of these strategies will have a distribution of objective values arising from pairing it with each of the N_1 attacking strategies, resulting in N_2 objective vectors. Based on the mean of these objective vectors, we identify the non-dominated (ND) strategies of the defender's population and calculate the normalized standard deviation of the defense's objective values for each ND strategy. The initial defense strategy is chosen as the ND strategy with the lowest average normalized standard deviation for the objective vectors arising from differences in pairing with the offense's strategies. Intuitively, the lowest normalized standard deviation means that it is most insensitive to the opponent's choice in its next move, making this defense strategy the most robust against uncertainties in the opponent's strategies.

Once the defense selects its strategy, the offense needs to choose its strategy from its own Pareto set. Every offense strategy is evaluated against the chosen defense strategy, resulting in N_1 offense objective vectors. These are checked for non-domination to each other, and the respective ND set is identified. To select a single offense strategy from the ND set, we use the highest trade-off-based selection method described in [37]. For any ND population, the trade-off value for an ND point x_i with a neighboring ND point x_j can be calculated as:

$$R(x_i, x_j) = \frac{w_{loss} \times Loss_f(x_i \to x_j)}{w_{gain} \times Gain_f(x_i \to x_j)}. \tag{7}$$

where w_{loss} and w_{gain} represent the weights given to the loss and gain in objectives for moving from x_i to x_j, respectively. This trade-off value provides an approximation of the amount of loss that can be tolerated, compared to the gain for moving from x_i to x_j. The final trade-off value for each strategy is computed by taking the average of its trade-off with its neighbors. A higher trade-off value indicates that choosing a neighboring strategy causes a high loss in one objective for a unit gain in another. Therefore, the highest

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2025.3565675

trade-off Pareto strategy is considered the best choice. Once this offense strategy is selected, the defender repeats the process of evaluating all its Pareto strategies against the chosen offense strategy and picks the one with the highest trade-off value. This process continues until an equilibrium strategy pair is reached, or an equilibrium cycle occurs, where the same sequence of attacking and defending strategies is repeatedly selected.

B. Decision-making using Progressive Shrinking with Co-evolutionary Optimization

In the previous approach, we assumed it was possible to move from one PO strategy to any other PO strategy in two subsequent iterations by an agent. However, due to the nature of wargames, once a strategy is selected, it may not be feasible to switch to any arbitrary strategy from its PO set. The chosen strategy might represent a specific deployment of resources following a grand plan, and the remaining PO strategies may not align with that plan, making a transition from one PO strategy to another illogical. To make the iterative DM process more practically viable, we need to use information about the variables and their ranges that remain viable and acceptable in subsequent iterations. While this requires substantial problem-specific knowledge, to demonstrate the idea of maintaining continuity among consecutive decisions, we use a simple approach described next.

After the defense and offense have selected their initial strategies, the first step is to fix the values of variables that should remain constant for smoother transitions, as determined by subject matter experts. These values are derived from the selected strategies of both the offense and defense. A separate MoCCoEv run is then conducted with a reduced number of variables, as some have been fixed. At this stage, the variation in each variable within its own PO set is analyzed. If the entire PO set for the defense exhibits minimal variation for certain variables, this indicates specific patterns among effective strategies for the defense. These variables, which remain nearly constant within the PO set, represent stable properties that must be kept fixed in subsequent strategies. To identify these low-variation variables, we compute the coefficient of variation (CoV) — the ratio of standard deviation to mean — for each defense variable in its PO set and select those with a CoV below a threshold value ($\tilde{\rho}$). These variables are then fixed at the values corresponding to the defense's selected strategy. The same procedure is applied to the offense. Not only does this process make the DM more pragmatic, but it also helps to reduce the number of variables in subsequent MoCCoEv iterations. A psuedo code for this iterative DM and progressive shrinking process is provided in Algorithm 2.

Other problem-specific variable shrinkage strategies can also be adopted for real-world scenarios, but the procedure outlined above provides a method to identify patterns of effective alternative strategies within an agent's PO set. It forces the agents to adhere to these patterns, progressively increasing variable shrinkage until no further patterns are detected within the chosen threshold. Once no more patterns are found, the variable shrinkage halts, and the DM approach described in the previous computational model is used to determine the final outcome of the game, resulting in either an equilibrium point or an equilibrium cycle.

Algorithm 2: A pseudo-code for the progressive shrinking-based co-evolutionary algorithm and iterative decision-making

Input: N_1, N_2, T, τ_1, τ_2, X_1, X_2, $\tilde{\rho}$
Output: Final populations Pop_1 and Pop_2
/* Initial MoCCoEv run and decision-making */
Pop_1, $Pop_2 \leftarrow$ MoCCoEv(X_1, X_2, N_1, N_2, T, τ_1, τ_2)
S_1, $S_2 \leftarrow$ DM(Pop_1, Pop_2)
$\rho_1 \leftarrow \mathrm{CoV}(Pop_1.Vars, \text{axis} = 0)$
$\rho_2 \leftarrow \mathrm{CoV}(Pop_2.Vars, \text{axis} = 0)$
$D \leftarrow |X_1| + |X_2|$
/* Progressively perform shrinking until the end criterion is met */
while $\min(\rho_1, \rho_2) < \tilde{\rho}$ **do**
  /* Shrink the variables by fixing their values */
  $\bar{X_1}, \bar{X_2} \leftarrow \phi, \phi$
  for $i = 1, \ldots, len(X_1)$ **do**
    if $\rho_1^i < \tilde{\rho}$ **then**
      $X_1^i = S_1^i$
      $\bar{X_1} \leftarrow \bar{X_1} \cup \{X_1^i\}$
  for $j = 1, \ldots, len(X_2)$ **do**
    if $\rho_2^j < \tilde{\rho}$ **then**
      $X_2^j = S_2^j$
      $\bar{X_2} \leftarrow \bar{X_2} \cup \{X_2^j\}$
  /* Reduce the number of variables and generations */
  $X_1 \leftarrow X_1 \setminus \bar{X_1}$
  $X_2 \leftarrow X_2 \setminus \bar{X_2}$
  $d \leftarrow |X_1| + |X_2|$
  $T \leftarrow (1 - \frac{d}{D}) \times T$
  /* Re-optimization and decision-making with the reduced variables */
  Pop_1, $Pop_2 \leftarrow$ MoCCoEv(X_1, X_2, N_1, N_2, T, τ_1, τ_2)
  S_1, S_2 = DM(Pop_1, Pop_2)
  /* Identify the next variable reductions */
  $\rho_1 \leftarrow \mathrm{CoV}(Pop_1.Vars, \text{axis} = 0)$
  $\rho_2 \leftarrow \mathrm{CoV}(Pop_2.Vars, \text{axis} = 0)$

V. Results and Discussion

After carefully formulating the proposed MoCCoEv optimization and DM procedures, we apply them to the WSOP. In this section, we first present the final formulation of the problem, followed by the results from the proposed progressive shrinking based co-evolutionary optimization and DM procedures. To better understand these results, it is essential to first revisit some of the findings from [24]. The initial part of the progressive shrinking method was introduced in [24], where MoCCoEv was applied to the WSOP. In this paper, we extend this approach by applying progressive shrinking to the results from the initial MoCCoEv run. For clarity and smoother flow, we first discuss some of the results presented in [24] and then focus on the new additions introduced in this work.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2025.3565675

A. Solution Evaluation and Surrogate Model

In our WSOP case study, each scenario is executed using the *Command Modern Air and Naval Operations* (CMANO) system. On average, each scenario takes approximately 6 minutes under a graphics-less accelerated execution. However, this duration is still too long to efficiently incorporate the high-fidelity model into an optimization run. To address this, surrogate models are developed for each of the competing attacker/defender objectives. A detailed description of the surrogate modeling process can be found in the supplementary material of this paper.

B. WSOP Objectives

The original set of objectives and goals of the WSOP agents are presented in Table I. Notice that the attacker and defender agents have opposite goals for every objective, making the WSOP a multi-objective, competitive, two-agent optimization problem. The correlation analysis of the dataset used for training the surrogate models reveals that some of the objectives are correlated with each other. We observed that *OffenseLifeCost* is highly correlated with *OffenseExpenditures* (correlation coefficient: 0.87) and similarly, *DefenseLifeCost* is highly correlated with *DefenseExpenditures* (correlation coefficient: 0.77). These observations are intuitive, as a higher *OffenseLifeCost* indicates more resources of the attacker are destroyed, leading to increased *OffenseExpenditures*. The same reasoning applies to the defending agent as well.

TABLE I: The original objectives and attacker-defender goals for the WSOP

Objective	Attacker Goal	Defender Goal
OffenseHits	↑	↓
OffenseLifeCost	↓	↑
DefenseLifeCost	↑	↓
OffenseExpenditures	↓	↑
DefenseExpenditures	↑	↓

For this reason, and to simplify the problem, we eliminate two objectives — *OffenseLifeCost* and *DefenseLifeCost* — from further consideration, as they are highly correlated with other objectives. This reduces the number of objectives to three. Additionally, we further reduce one objective by considering the difference in the expenditures, as presented in Table II.

TABLE II: Reduced objectives for WSOP

i	Objective (f_i)	Attacker Goal	Defender Goal
1	*OffenseExpenditures - DefenseExpenditures*	↓	↑
2	*OffenseHits*	↑	↓

C. Multi-objective Competitive Co-evolutionary (MoCCoEv) Optimization

1) Competitive Trade-off Solutions: We use a population size of 50 for each co-evolving population and run the proposed MoCCoEv algorithm for 200 generations. The other parameters are kept the same as before. The final outcome of the MoCCoEv algorithm consists of two ND sets of solutions (each having 10 solutions), one for each agent. We compute average objective values for each solution for one agent with every member of the other agent and then identify the ND solutions for both agents based on these average values. The respective ND fronts based on these average objective vectors of each agent are plotted in Figure 2. Clearly, for the offense population, objectives (*OffenseExpenditures* − *DefenseExpenditures*) and −*OffenseHits* are minimized, while for the defense population, the negatives of these objectives are minimized.

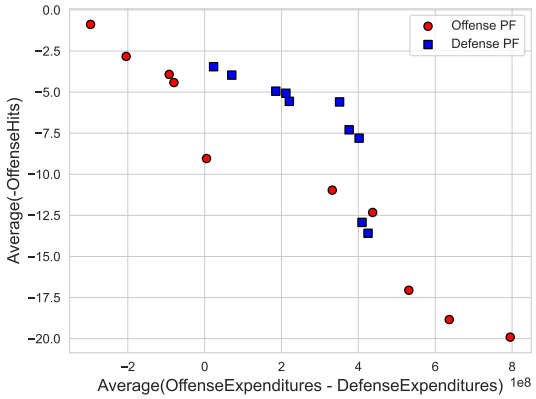


Fig. 2: Offense and Defense ND sets obtained by MoCCoEv (Taken from [24])

In all ND solutions for the defense, offense expenditure exceeds that of defense. To achieve up to 10 offense hits, the offense needs to spend slightly more than the defense. However, to destroy more than 10 assets of the defense, the offense must outspend the defense. For example, to achieve 20 offense hits, the offense needs to spend about 8 million units more than the defense. These results are very different from the results that we get when we independently optimize the offense and defense objectives. Please refer to the supplementary material to check the results for the independently optimized objectives for offense and defense.

Interestingly, if the offense is willing to cause an average damage of about 2.5 units, it needs to spend 2 million units less than the defense. Additionally, the ND solutions for the offense exhibit a wider range of objective values than those for the defense.

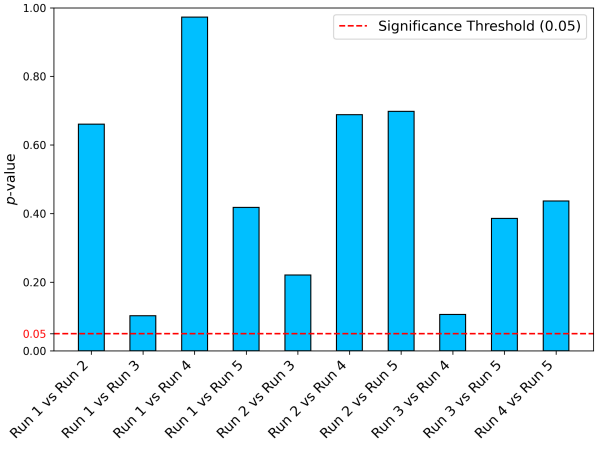
These observations are significant, providing both offense and defense agents with better insights into various possible alternative scenarios before they prepare to launch any action. In this paper, we focus on the solution approach, its flexibility, and its potential in offering a better understanding of wargame scenarios — a factor that we believe holds tremendous value in real-world applications.

2) Statistical Significance Testing for MoCCoEv Runs: To assess whether the results of the MoCCoEv runs vary statistically significantly, we conducted 5 MoCCoEv runs and compared the final offense and defense strategies using the Wilcoxon rank-sum test (or Mann-Whitney U-test) [38] at a significance level of 0.05. The Wilcoxon rank-sum test is a non-parametric statistical test used to compare two indepen-

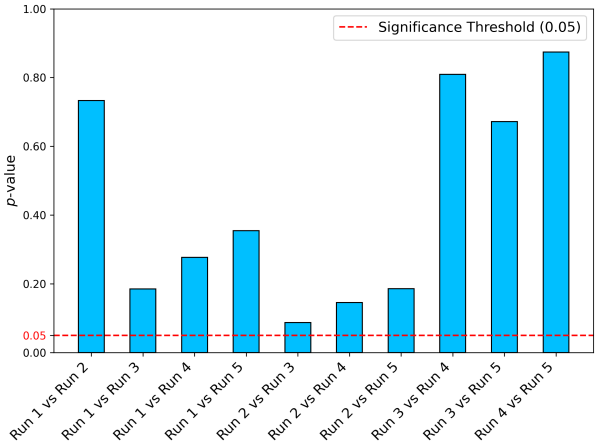
This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2025.3565675

dent sets of samples. The final results of these comparisons are presented in Figure 3 for both the offense and defense strategies. From the analysis, it is evident that the test could not reject the null hypothesis, as the p-values in all cases are greater than 0.05. This suggests that the samples are drawn from the same distribution, indicating that there is no statistically significant difference between the results of the multiple MoCCoEv runs. This confirms the robustness of the MoCCoEv process.



(a) Pairwise Wilcoxon rank-sum test comparison for p-values of offense strategies over multiple MoCCoEv runs



(b) Pairwise Wilcoxon rank-sum test comparison for p-values of defense strategies over multiple MoCCoEv runs

Fig. 3: Statistical significance test outcome for multiple MoCoEv runs

3) Optimization With and Without Co-evolution: Figure 4 offers a comprehensive comparison of our optimizations thus far. The original high-fidelity objective vectors are shown as yellow circles and were used to construct the surrogate models for each objective. Note that the objective *OffenseHits* is now shown on the y-axis, instead of its negative value, placing the best offense strategies in the top-left corner of the plot, and the best defense strategies in the bottom-right corner.

The data are centered around equal expenditures for both agents (x-axis value of zero), causing a median around four offense hits. Let us assume we do not employ any optimization

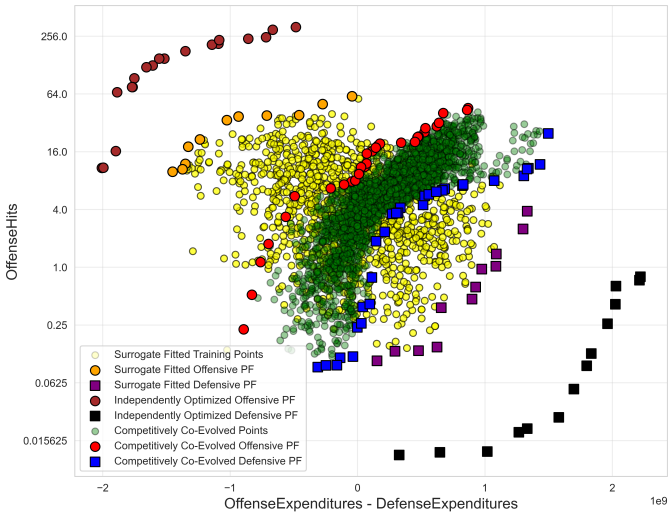


Fig. 4: Training data, independently optimized strategies, and MoCCoEv strategies (taken from [24])

algorithm and try to extract the best offense and defense strategies from the initial samples. For offense, we find that the respective ND front lies along the top-left boundary of the sample points (yellow circles), marked by orange circles. Similarly, the best strategies for defense are located in the training dataset at the bottom-right corner of the yellow circles, marked with violet squares. These fronts of solutions are still widely different from each other and ignore the effects of one agent's strategies on the other.

Next, we plot the final MoCCoEv ND solutions for both agents in red circles and blue squares. As we set a population size of 50 for each agent, we end up with $50 \times 50 = 2,500$ strategy pairs, represented by the green-colored open circles. It is clear that the co-evolutionary solution sets are closer to each other, recognizing one agent's ability to influence the other. The MoCCoEv algorithm successfully brings the ND fronts of each agent closer to one another, showing the role of our optimization algorithm in identifying new, competitive solutions (green) compared to the initial samples (yellow).

In a multi-agent system with multiple conflicting criteria for each agent, MoCCoEv is expected to find a set of Pareto optimal solutions for each agent. This can be achieved offline by optimizing models derived from case studies or training data collected in the past. Once these Pareto sets are obtained for each agent, they can select an appropriate solution from their respective Pareto sets to play the game. By doing so, each agent ensures that they are always picking strategies from their best possible sets. Before we demonstrate how a DM task helps choose strategies from these Pareto sets, we discuss a task used to develop our proposed progressive shrinking approach.

4) "Innovization": Extracting Common Patterns in Trade-off Solutions: The process of extracting common patterns in a set of ND solutions is termed "innovization" [39]. Patterns can be extracted from the ND solutions manually, using certain problem knowledge [39], or through an automated machine learning process [40]. Here, we employ a manual process in which all 74 variables of both agents are plotted in order

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2025.3565675

of their similarity across the obtained ND solutions. For this purpose, we compute the coefficient of variation (CoV = σ/μ) for each variable and order them in decreasing CoV value for offense and defense variables independently, as shown in Figure 5.

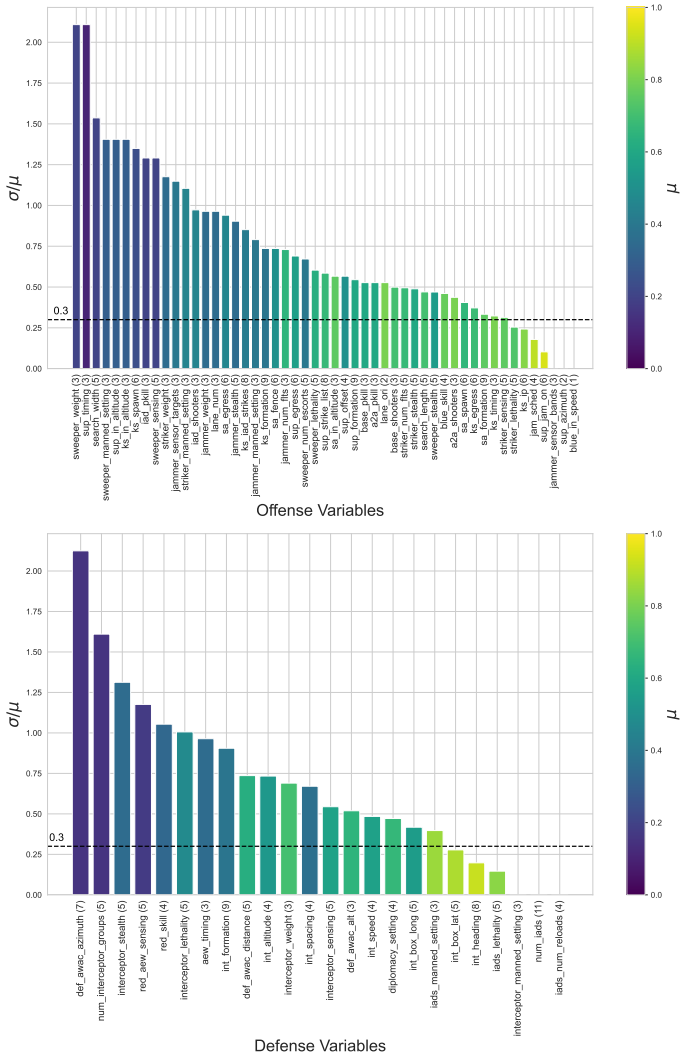


Fig. 5: CoVs for offense and defense variables indicate convergence of certain variables across ND solutions (taken from [24])

A higher CoV indicates greater dispersion around the mean, while a lower CoV reflects convergence towards the mean value in the entire ND solution set. It is reported [41] that a threshold of $\sigma/\mu = 0.3$ is acceptable for assuming good convergence. Using this threshold, we observe that seven of the 50 offense variables and six of the 24 defense variables are well-converged. The number of possible values each variable can take in the original formulation is mentioned in parentheses in Figure 5. Additional analysis of these 13 converged variables is provided in the supplementary document.

5) Multi-Criterion Decision Making on MoCCoEv Solutions: Finding a set of ND solutions for each agent in a co-evolutionary process is a challenging computational task, but it is only part of the overall WSOP task. The next important task is to select preferred solutions for implementation. Despite numerous MCDM studies in the literature [30], [31], MCDM studies for two co-evolved and interacting Pareto sets are missing. In this paper, we propose a viable MCDM procedure.

Since the two agents — defense and offense — are independent, they are likely to make decisions independently of each other. However, the linkage of the two agents in defining the objectives suggests that a sequential DM process should alternate between the two. This results in a defender-attacker simulation game, beginning with the defense declaring its strategy. The offense then selects its strategy to counter the declared defense strategy. After the offense's selection, the defense makes the next move to select its most appropriate response. This iterative process can continue until an equilibrium pair of strategies or an equilibrium cycle of strategies is reached. We have outlined two DM approaches in Section IV.

As outlined in Section IV, we first identify the specific defense strategy corresponding to the smallest standard deviation in its objective vectors arising from various combinations of strategies in the offense population. Figure 6 shows the size of each defense strategy in proportion to the corresponding standard deviation. Strategy 9 (second from the top-left point) produces the least standard deviation when combined with each offense ND strategy. Hence, *Defense_9* is selected as the preferred initial defense strategy.

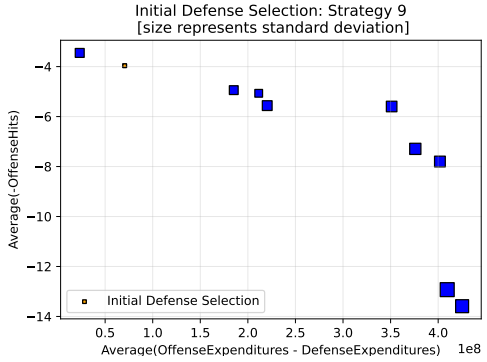


Fig. 6: Selection of initial defense strategy (taken from [24])

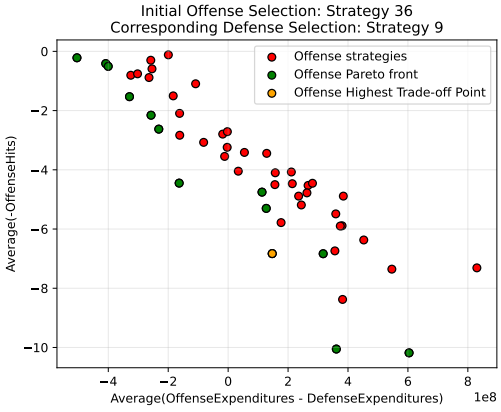


Fig. 7: Selection of initial offense strategy (taken from [24])

After selecting the initial defense strategy, we now plot the objective vector of every offense strategy from the final

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2025.3565675

50 population of offense strategies with this defense strategy (*Defense_9*) in Figure 7. We now identify the non-dominated set from these vectors and select the highest trade-off point. The strategy *Offense_36*, marked with a yellow circle, is the best response to *Defense_9*. This strategy makes the highest loss in one objective for a unit gain in the other, with no motivation to move to a neighboring strategy. Hence, *Offense_36* is the preferred strategy for offense. Next, the defense selects its best strategy from its own MoCCoEv-obtained set. With a similar trade-off analysis of all offense strategies, we find that *Defense_43* is the best response. Continuing the DM process reveals that *Defense_9* is now the best defense strategy. Interestingly, this leads to a cycle of strategies, ending in an equilibrium cycle. Figure 8 shows the cycle of strategies.

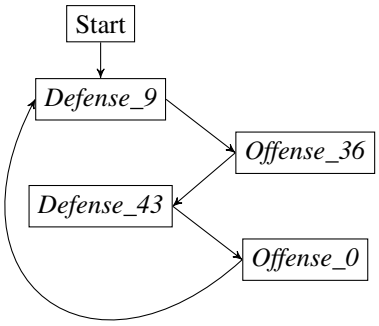


Fig. 8: Proposed MCDM scheme results in an equilibrium cycle of strategies: $D_9 \rightarrow O_{36} \rightarrow D_{43} \rightarrow O_0 \rightarrow D_9$ (taken from [24])

D. Progressive-shrinking based Optimization Results

In the previous experiments with the co-evolutionary optimization and offline decision-making, we observed that some of the variables exhibit a small coefficient of variation in the co-evolved populations, as shown in Figure 5. Moreover, as discussed in Section IV-B, certain variables need to be fixed for a smooth transition from one iteration to the next. To address these issues, we apply the progressive shrinking methodology. Initially, we start from the set of combined strategies obtained at the end of the co-evolutionary process, marked as "Competitively Co-Evolved Points" in Figure 4.

The first step of the progressive shrinking approach is to fix the expert-guided variables to the values of the same variables in the point marked as "DM Selection Level 1" in Figure 11a. After fixing these variables, we re-optimize using an EMO (NSGA-II) algorithm to obtain a new set of strategies, marked as "Progressive Shrinking Round 1" in Figure 11b. The CoVs for all variables after this run are shown in Figures 12a and 12c.

After evaluating the variables against a threshold of $\rho = 0.3$, we observe some variables have CoVs less than 0.3. In the next iteration of the progressive shrinking process, these variables are fixed to the values of the next decision-making point. This procedure continues until no more variables with CoVs less than the threshold remain. By applying this approach, we perform eight consecutive progressive shrinking runs, resulting in a population of strategies where all variables have CoV greater than ρ. The final strategies are shown in Figure 11c, and the CoVs for all variables are presented in Figures Figure 12b and 12d. The collection of strategies after each progressive shrinking run is also illustrated in Figure 13. From the figure, we cannot identify a clear path of interest, but by comparing the first decision point and the final one, we observe that the final decision point has shifted primarily horizontally, favoring the offense's expenditure, while keeping offense hits around 8 units.

Starting with 50 variables for offense and 24 variables for defense, we end up with only eight variables for offense and four variables for defense after eight consecutive progressive shrinking runs. This strategy is more practical, as it mimics the typical scenario in wargames where variables are progressively fixed over time due to earlier commitments.

At the end of the progressive shrinking process, we arrive at the yellow points shown in Figure 11c. A final level of decision-making is then performed to select one of the strategy combinations from the yellow points. Following the same DM process outlined in Section V-C5, we select *Defense_0* as the first strategy chosen by the defense (shown in Figure 9), and *Offense_11* as the counter-strategy selected by the offense (shown in Figure 10). This decision point is marked in the objective space in Figure 11c with a red cross. All 74 variable values for this decision point are shown in Figure 15 in a PCP plot.

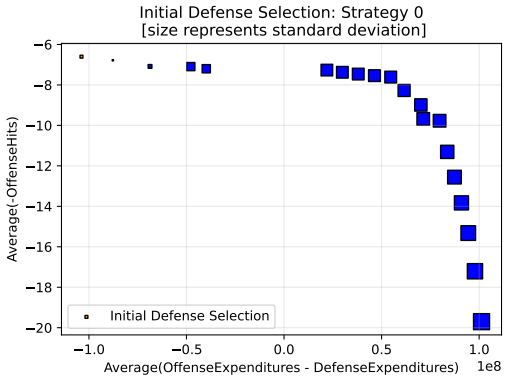


Fig. 9: Selection of initial defense strategy in the progressive shrinking-based MoCCoEv Pareto front

Finally, the DM process continues, resulting in an equilibrium point between *Offense_11* and *Defense_14*, as shown in Figure 14.

E. Human Expert's Interpretation of Equilibrium Strategies

A careful review of Figure 15 by a wargame expert reveals the following details about the offense's and defense's strategies. The attacker's strategy uses many large and exquisite strikers with many missiles. These are protected by a few small and well-equipped anti-air sweepers combined with four well-equipped electronic attack (EA) assets with powerful jammers. All assets leverage modern stealth technology to minimize

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2025.3565675

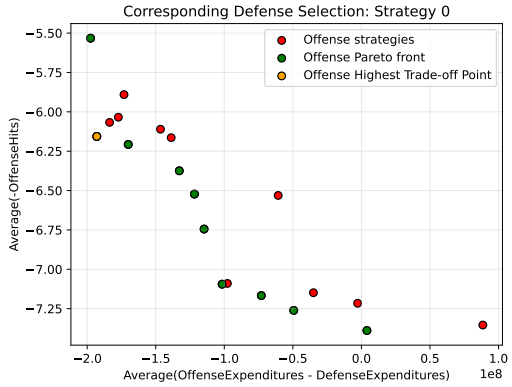
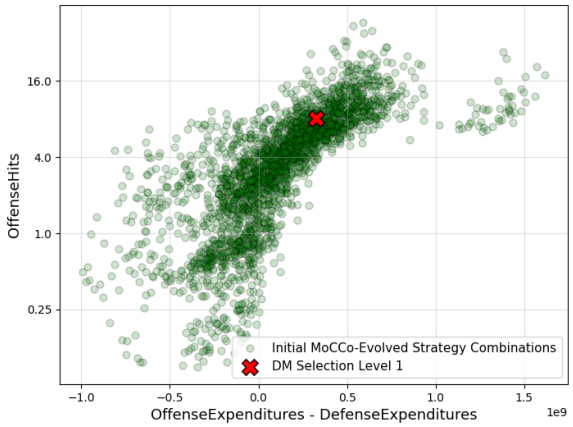


Fig. 10: Initial offense strategy selection (*Offense_11*) in the progressive shrinking-based MoCCoEv Pareto front

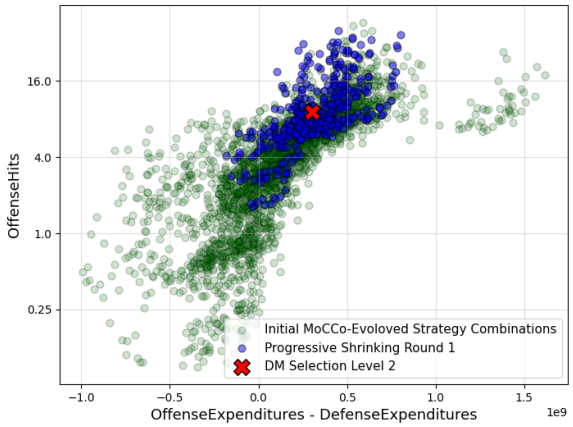
their detection signatures. Only coordinated strikes with many shooters are allowed, after which if ordinance remained, it is used to attack surface-to-air missile systems. The offense layers their assets with the strikers coming in low at 15,000 ft, EA assets at 30,000 ft, and anti-air at 35,000 ft. This makes the strikers harder to detect and maximizes the *energy advantage* of anti-air missiles. The attacker strike package flies from the west to the east with EA assets trailing the strike assets to the west, thereby providing good protection of the strike angles whose formation angled the assets toward the east. Additionally, it provides good jamming angles, and due to the increased distance traveled, may have been done to lure defensive interceptors outside the protective zone of their surface to air missile systems. EA assets are very aggressive using long range jamming and aggressively attacking detected surface to air missile systems.

The defender strategy focuses heavily on long range aerial early warning to support an extensive and heavily equipped air defense system. Maximizing the number of surface to air missile systems and their supply of missiles, the aerial warning system is launched on initial detects of early warning RADAR to provide a better air picture to the air defense missile batteries. A single large but very stealthy interceptor is then launched to support the ground missile batteries.

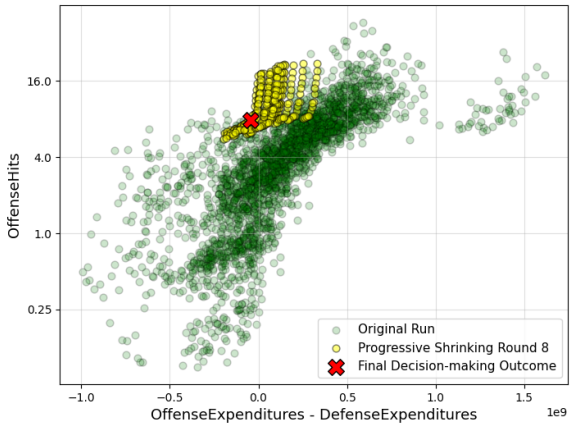
In a discussion with real wargame experts, these strategies are considered smart and intelligent. Interestingly, they are solely obtained by our MoCCoEv and DM algorithms and without explicitly using any real human decision-makers.



(a) DM selection before the first progressive shrinking run



(b) Strategies after the first progressive shrinking run



(c) Strategies after the final progressive shrinking run

Fig. 11: Different stages of progressive shrinking run

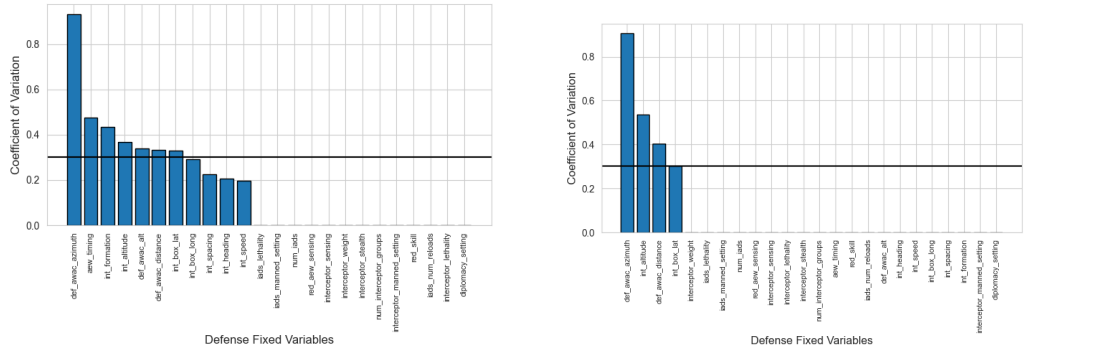
VI. Conclusions and Future Studies

In this paper, we have addressed a real-world wargame strategy optimization problem using a multi-objective, competitive, co-evolutionary (MoCCoEv) algorithm. The co-evolutionary approach has integrated the well-established NSGA-II procedure to generate two sets of non-dominated (ND) strategies for the offense and defense agents. We proposed two DM procedures, beginning with a preferred (most robust) strategy from the defense's ND front. Subsequently, the offense and defense agents alternately select the best trade-off strategies from their respective ND fronts in response to the opponent's chosen strategy. We compared the MoCCoEv results with ND solutions obtained without optimization and with single-agent optimization, providing a detailed analysis of the final strategies and deriving valuable insights into the problem variables. Our findings have demonstrated that the final outcome can either be an equilibrium offense-defense strategy or an equilibrium cycle of offense-defense strategies.

This study highlights the potential of multi-objective, co-evolutionary optimization using EMO algorithms and opens several avenues for future research and applications. Future

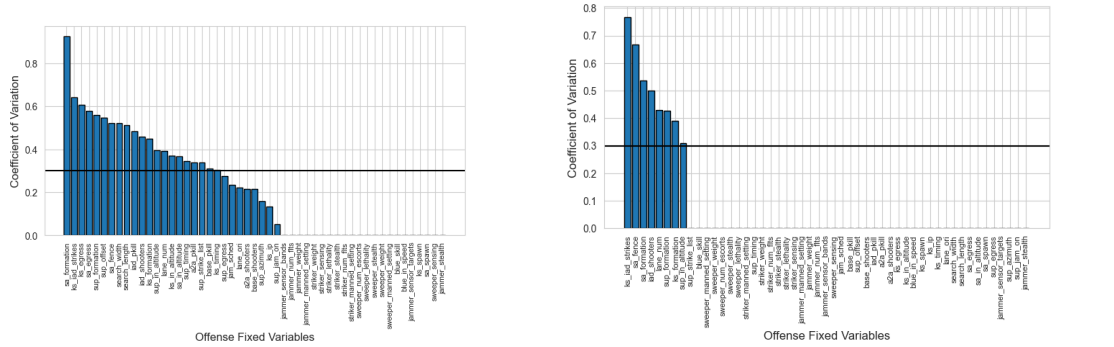
This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2025.3565675



(a) CoV of Defense variables after first progressive shrinking run

(b) CoV of Defense variables after final progressive shrinking run



(c) CoV of Offense variables after first progressive shrinking run

(d) CoV of Offense variables after final progressive shrinking run

Fig. 12: CoVs after the first and the final progressive shrinking run

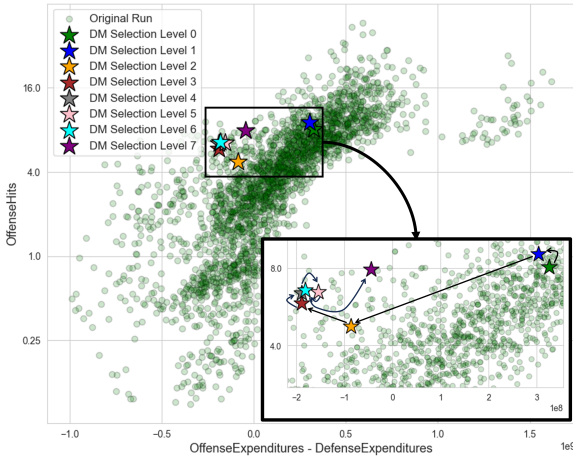


Fig. 13: Movement of the DM point in the search space over consecutive progressive shrinking runs

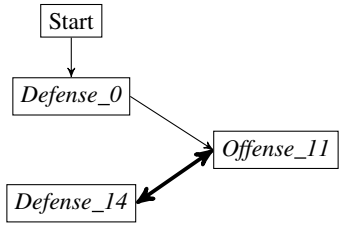


Fig. 14: Proposed progressive shrinking strategy results in an equilibrium point: $D_0 \rightarrow O_{11} \rightarrow D_{14} \rightarrow O_{11}$ (2-sided bold arrow)

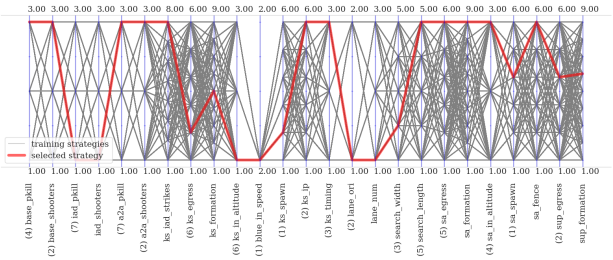
work should extend this approach to problems with more than two objectives and explore other multi-objective optimization algorithms. Additionally, instead of using average objective values from the opponent's population as fitness, alternative statistical metrics, such as standard deviation, could be considered to achieve more robust Pareto optimal fronts. Further pragmatic DM procedures should also be explored.

In this work, we have primarily focused on two distinct types of computational models for addressing the WSOP. However, a third category of computational models may offer greater practical applicability. Although progressive shrinking incorporates a pragmatic DM approach, decisions are made for both agents sequentially, without re-optimizing one agent's strategies based on the other agent's decision. Instead, strategies for both agents are selected from their respective strategy sets, which are obtained through optimization.

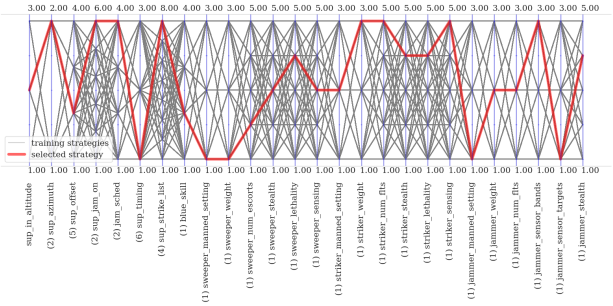
An alternative approach could involve making optimization and DM more practical. After an agent chooses its preferred strategy, the opponent agent could employ an evolutionary (non-co-evolutionary) multi-objective optimization (EMO) procedure with a reduced set of variables determined by the opponent's previously chosen strategy. The EMO procedure would identify the best possible Pareto strategies for the opponent that can be transitioned to. A DM approach would

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

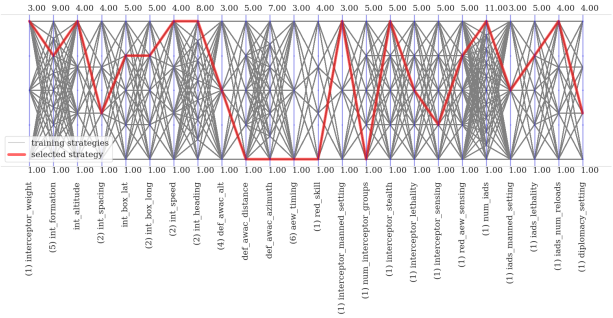
This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2025.3565675



(a) PCP of the first 24 variables of offense



(b) PCP of the next 26 variables of offense



(c) PCP of all 24 variables of defense

Fig. 15: PCP for 74 variables of the final solution (colored in red) selected by the DM process in the final run of the progressive shrinking algorithm. The training points used to train the surrogate models are also shown in grey for comparison. The brackets in front of some of the variables represent the iteration in which they got fixed during progressive shrinking

then be used to select the most preferred Pareto strategy in response to the opponent's declared strategy. This process would be repeated by each agent in tandem, with the EMO optimization performed online to generate the best Pareto strategies within the shrinking search space.

This iterative single-agent optimization and DM procedure could continue until a stopping criterion is met. We did not implement this concept in the current study to maintain the focus on MoCCoEv optimization, but we plan to pursue this approach in future research. Despite these opportunities for further refinement, this study has clearly demonstrated the utility of co-evolution, multi-objective optimization, and multi-agent DM with multiple ND fronts, paving the way for further future investigations in this area.

ACKNOWLEDGEMENT

This research was supported by Northrop Grumman Corporation as a part of a collaboration between Computational Optimization and INnovation (COIN) Lab at Michigan State University and Northrop Grumman.



Ritam Guha (Student Member, IEEE) received a Bachelor's degree in Computer Science and Engineering from Jadavpur University, India in 2020. He started his Ph.D. at Michigan State University in 2021. His research interests are at the intersection of evolutionary computation, multi-objective optimization, co-evolutionary optimization, machine learning, and interpretable AI. During his Ph.D., he has worked on multiple industrial collaborations (with Ford Motor Company, Hemlock Semiconductor, Northrop Grumman, and Google) and academic projects, addressing important aspects of AI, wargaming, and manufacturing problems.



Ryan McKendrick Dr. Ryan McKendrick is a Staff Cyber Systems Engineer and Technical Fellow at Northrop Grumman Mission Systems. He received a Ph.D. from George Mason University in Human Factors and Applied Cognition. Dr. McKendrick has publications exploring the neuroergonomics of mobile displays, the interaction between distractions, mobility, and cognitive work, the neurovascular effects of noninvasive brain stimulation, as well as cognitive load classification. His current work focuses on human-machine multi-objective decision making, parsimonious modeling, and dynamics discovery.



Bradley Feest Bradley Feest received Bachelor's and Master's degrees in Electrical Engineering from the University of California, San Diego, where he focused on machine learning and signal processing. He has worked in Artificial Intelligence research at Northrop Grumman since 2014, as a software developer and data scientist. His recent research uses Genetic Programming and Sparse Regression to model complex battlespaces.



Kalyanmoy Deb (Fellow, IEEE) received the Bachelor's degree in Mechanical Engineering from the Indian Institute of Technology Kharagpur, India in 1985 and the master's and Ph.D. degrees from the University of Alabama, Tuscaloosa, USA in 1989 and 1991, respectively. He is a University Distinguished Professor and Koenig Endowed Chair Professor with the Department of Electrical and Computer Engineering, Michigan State University, East Lansing, USA. His research interests are in evolutionary optimization and their application in multi-criterion optimization, modeling, and machine learning. He has published over 620 research papers with Google Scholar citations of over 218,000 and an h-index of 146. Prof. Deb is a Fellow of IEEE, ACM, and ASME. He was awarded the IEEE Evolutionary Computation Pioneer Award, Infosys Prize, and the CajAstur Mamdani Prize, among others.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2025.3565675

REFERENCES

[1] G. M. De Lima Filho, A. R. Kuroswiski, F. L. L. Medeiros, M. Voskuijl, H. Monsuur, and A. Passaro, "Optimization of unmanned air vehicle tactical formation in war games," *IEEE Access*, vol. 10, pp. 21 727–21 741, 2022.

[2] Z.-X. Jia and J.-F. Kiang, "War game between two matched fleets with goal options and tactical optimization," *AI*, vol. 3, no. 4, pp. 890–930, 2022.

[3] A. Ozaki, M. Furuichi, K. Takahashi, and H. Matsukawa, "Design and implementation of parallel and distributed wargame simulation system and its evaluation," *IEICE TRANSACTIONS on Information and Systems*, vol. 84, no. 10, pp. 1376–1384, 2001.

[4] J. Luo, J. Cooper, C. Cao, and K. Pham, "Cooperative adaptive control of a two-agent system," in *2012 American Control Conference (ACC)*. IEEE, 2012, pp. 2413–2418.

[5] Y. Li, J. Wang, and Z. Liu, "A simple two-agent system for multi-objective flexible job-shop scheduling," *Journal of Combinatorial Optimization*, vol. 43, no. 1, pp. 42–64, 2022.

[6] K. Wang, C. Gou, Y. Duan, Y. Lin, X. Zheng, and F.-Y. Wang, "Generative adversarial networks: introduction and outlook," *IEEE/CAA Journal of Automatica Sinica*, vol. 4, no. 4, pp. 588–598, 2017.

[7] E. Russell, "Coevolutionary history," *The American Historical Review*, vol. 119, no. 5, pp. 1514–1528, 2014.

[8] N. Keerativuttitumrong, N. Chaiyaratana, and V. Varavithya, "Multi-objective co-operative co-evolutionary genetic algorithm," in *International Conference on Parallel Problem Solving from Nature*. Springer, 2002, pp. 288–297.

[9] A. Atashpendar, B. Dorronsoro, G. Danoy, and P. Bouvry, "A scalable parallel cooperative coevolutionary pso algorithm for multi-objective optimization," *Journal of Parallel and Distributed Computing*, vol. 112, pp. 111–125, 2018.

[10] C. K. Goh, K. C. Tan, D. Liu, and S. C. Chiam, "A competitive and cooperative co-evolutionary approach to multi-objective particle swarm optimization algorithm design," *European Journal of Operational Research*, vol. 202, no. 1, pp. 42–54, 2010.

[11] I. R. Meneghini, F. G. Guimarães, and A. Gaspar-Cunha, "Competitive coevolutionary algorithm for robust multi-objective optimization: The worst case minimization," in *2016 IEEE Congress on Evolutionary Computation (CEC)*, 2016, pp. 586–593.

[12] L. Miguel Antonio and C. A. Coello Coello, "Coevolutionary multi-objective evolutionary algorithms: Survey of the state-of-the-art," *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 6, pp. 851–865, 2018.

[13] L. Corbo, S. Kraus, B. Vlačić, M. Dabić, A. Caputo, and M. M. Pellegrini, "Coopetition and innovation: A review and research agenda," *Technovation*, vol. 122, p. 102624, 2023.

[14] D. E. Goldberg, *Genetic Algorithms for Search, Optimization, and Machine Learning*. Reading, MA: Addison-Wesley, 1989.

[15] J. Paredis, "Coevolutionary constraint satisfaction," in *Parallel Problem Solving from Nature III (PPSN-III)*, 1994, pp. 46–55.

[16] H. J. C. Barbosa, "A genetic algorithm for min-max problems," in *Proceedings of the First International Conference on Evolutionary Computation and Its Application (EvCA'96)*, 1996, pp. 99–109.

[17] S. G. Ficici and J. B. Pollack, "Pareto optimality in coevolutionary learning," in *European Conference on Artificial Life*. Springer, 2001, pp. 316–325.

[18] E. D. D. Jong and J. B. Pollack, "Ideal evaluation from coevolution," *Evolutionary computation*, vol. 12, no. 2, pp. 159–192, 2004.

[19] A. Benford, M. Olhofer, T. Rodemann, and P. K. Lehre, "Bicriteria optimisation of average and worst-case performance using coevolutionary algorithms," in *IEEE Congress on Evolutionary Computation (IEEE CEC) 2024*. IEEE, 2024.

[20] P. K. Lehre and S. Lin, "Towards runtime analysis of population-based co-evolutionary algorithms on sparse binary zero-sum games," in *The 39th Annual AAAI Conference on Artificial Intelligence*. AAAI Press, 2025.

[21] K.-B. Sim and J.-Y. Kim, "Solution of multiobjective optimization problems: coevolutionary algorithm based on evolutionary game theory," *Artificial Life and Robotics*, vol. 8, pp. 174–185, 2004.

[22] A. Suresh, K. Deb, and V. N. Boddeti, "Towards multi-objective co-evolutionary problem solving," in *Proceedings of the 11th International Conference om Evolutionary Multi-Criterion Optimization (EMO-2021)*. Springer, 2021, pp. 139–151.

[23] A. Suresh, J. Kongmanee, K. Deb, and V. N. Boddeti, "Multi-objective coevolution and decision-making for cooperative and competitive environments," in *2021 IEEE Congress on Evolutionary Computation (CEC-2021)*. IEEE, 2021, pp. 1601–1608.

[24] R. Guha, R. Mckendrick, B. Feest, and K. Deb, "Attacker-defender strategy optimization using multi-objective competitive co-evolution," in *International Conference on Parallel Problem Solving from Nature*. Springer, 2024, pp. 351–366.

[25] P.-I. Evensen, S. E. Martinussen, M. Halsør, and D. H. Bentsen, "Wargaming evolved: Methodology and best practices for simulation-supported wargaming," in *The Interservice/Industry Training, Simulation & Education Conference (I/ITSEC), Volume 2019*, 2019.

[26] P. J. Schwartz, D. V. O'Neill, M. E. Bentz, A. Brown, B. S. Doyle, O. C. Liepa, R. Lawrence, and R. D. Hull, "Ai-enabled wargaming in the military decision making process," in *Artificial intelligence and machine learning for multi-domain operations applications II*, vol. 11413. SPIE, 2020, pp. 118–134.

[27] R. Guha and K. Deb, "Compromising pareto-optimality with regularity in platform-based multi-objective optimization," *IEEE Transactions on Evolutionary Computation*, 2023.

[28] R. Guha and K. Deb, "RegEMO: Sacrificing pareto-optimality for regularity in multi-objective problem-solving," in *International Conference on Evolutionary Multi-Criterion Optimization*. Springer, 2023, pp. 29–42.

[29] R. Guha and K. Deb, "Scalable polynomial RegEM(a)O for multi-/many-objective platform-based design optimization problems," in *2024 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2024, pp. 1–8.

[30] K. Miettinen, *Nonlinear Multiobjective Optimization*. Boston: Kluwer, 1999.

[31] R. E. Steuer, *Multiple Criteria Optimization: Theory, Computation and Application*. New York: Wiley, 1986.

[32] K. Deb, *Multi-objective optimization using evolutionary algorithms*. Chichester, UK: Wiley, 2001.

[33] P. Shukla and K. Deb, "Comparing classical generating methods with an evolutionary multi-objective optimization method," in *Proceedings of the Third International Conference on Evolutionary Multi-Criterion Optimization (EMO-2005)*, 2005, pp. 311–325, lecture Notes on Computer Science 3410.

[34] W. Deng, X. Cai, D. Wu, Y. Song, H. Chen, X. Ran, X. Zhou, and H. Zhao, "Moqea/d: Multi-objective qea with decomposition mechanism and excellent global search and its application," *IEEE Transactions on Intelligent Transportation Systems*, 2024.

[35] C. Huang, D. Wu, X. Zhou, Y. Song, H. Chen, and W. Deng, "Competitive swarm optimizer with dynamic multi-competitions and convergence accelerator for large-scale optimization problems," *Applied Soft Computing*, vol. 167, p. 112252, 2024.

[36] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: Nsga-ii," *IEEE transactions on evolutionary computation*, vol. 6, no. 2, pp. 182–197, 2002.

[37] S. Mittal, D. Kumar, and S. K. Deb, "A unified automated innovization framework using threshold-based clustering," in *2020 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2020, pp. 1–8.

[38] G. W. Corder and D. I. Foreman, *Nonparametric statistics: A step-by-step approach*. John Wiley & Sons, 2014.

[39] K. Deb and A. Srinivasan, "Innovization: Innovating design principles through optimization," in *Proceedings of the 8th annual conference on Genetic and evolutionary computation*, 2006, pp. 1629–1636.

[40] S. Bandaru, A. H. C. Ng, and K. Deb, "Data mining methods for knowledge discovery in multi-objective optimization: Part a – survey," *Expert Systems With Applications*, vol. 70, pp. 139–159, 2017.

[41] Coefficient of variation. Accessed on September 27, 2023. [Online]. Available: https://www.isixsigma.com/dictionary/coefficient-of-variation/

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/